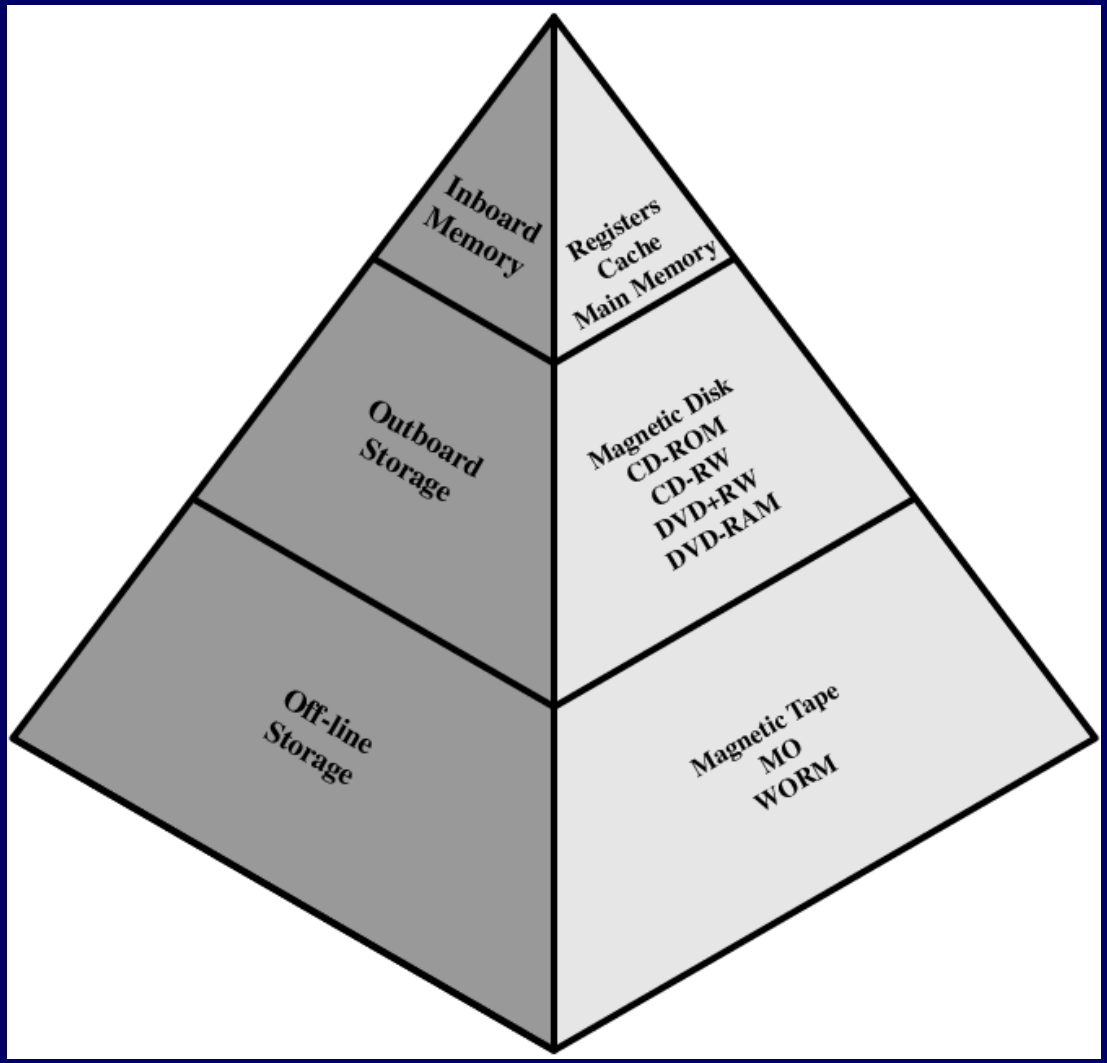
Computer Organisation

Topic 11
Cache Memory

Outline

- Cache Memory Principles
- Elements of Cache Design
  - Mapping function
  - Replacement algorithms
- Pentium 4/PowerPC caches

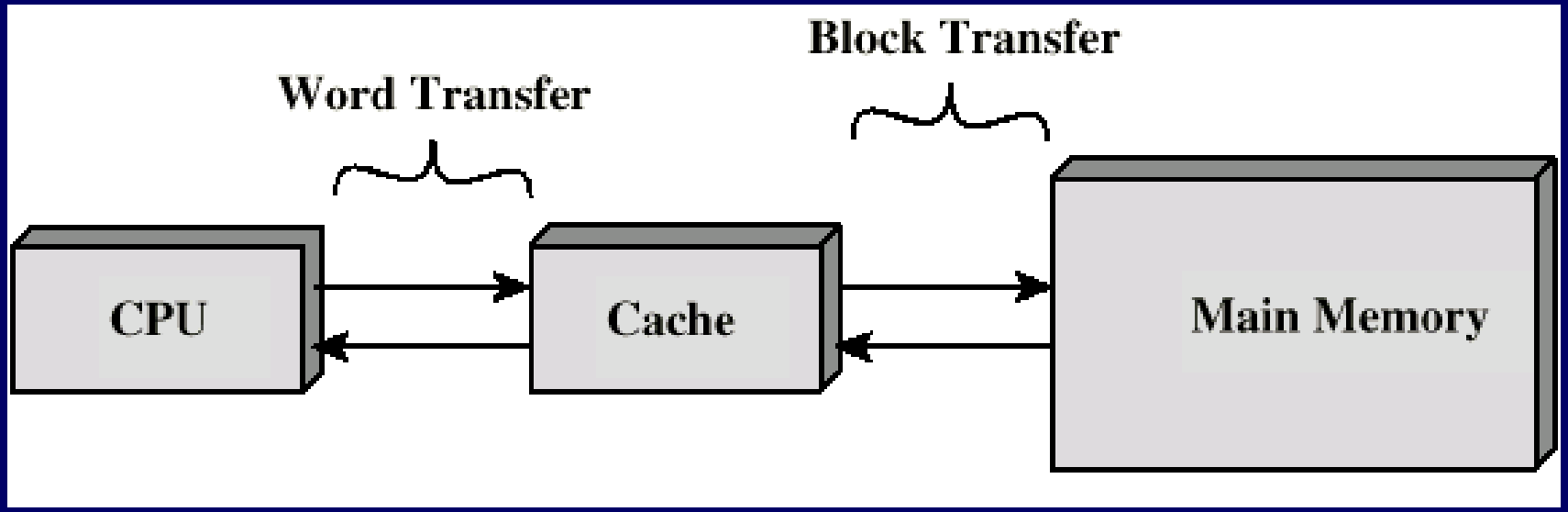
Memory Hierarchy - Diagram



Cache

- Small amount of fast memory
  - While at same time providing as much memory as possible at the price of cheaper semiconductor memories
- Sits between normal main memory and CPU
  - Main memory is large but slow
  - Cache memory is small but fast
- May be located on CPU chip or module
- Cache contains copy of portions of main memory

Cache

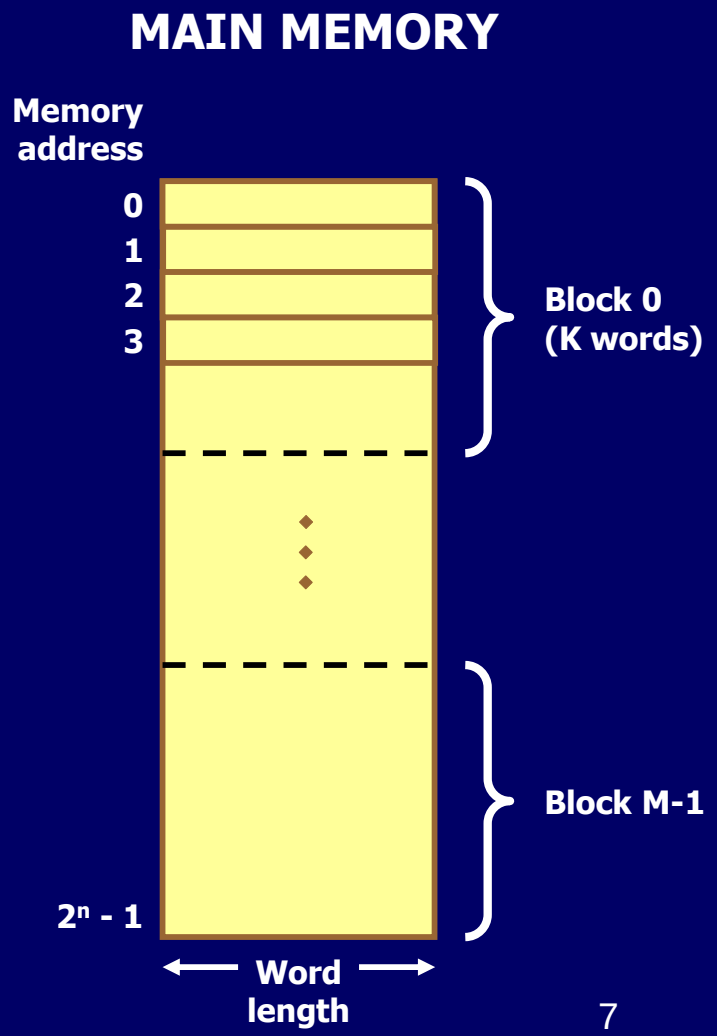


Cache Operation - Overview

- CPU requests contents of memory location
- Check cache for this data
  - If present, get from cache (fast)
  - If not present, read required block from main memory to cache
    - Then deliver from cache to CPU
- Locality of reference
  - When a block of data is fetched into cache to satisfy a single memory reference, it is likely there will be future reference to that same location

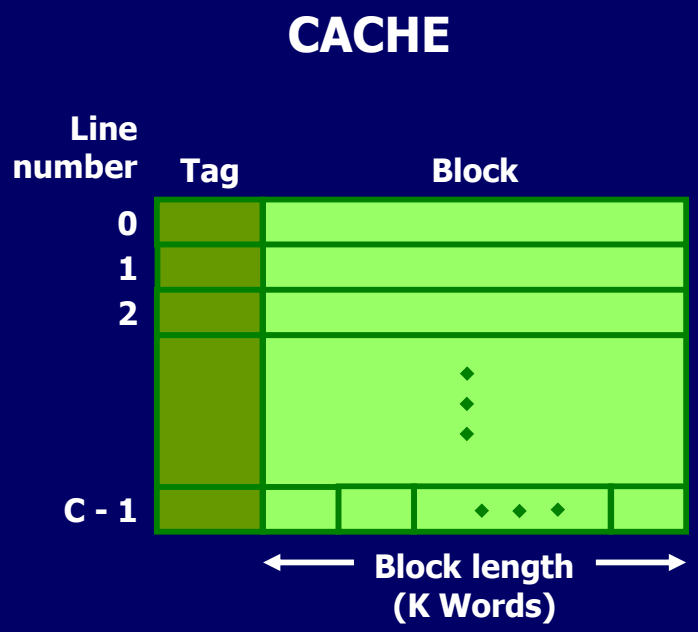
Cache/Main Memory Structure

- Main memory: 2^n addressable words
  - Each word has unique n bit address
- Main memory considered to consist of M number of blocks
  - Each block has K words
  - M = 2^n / K blocks

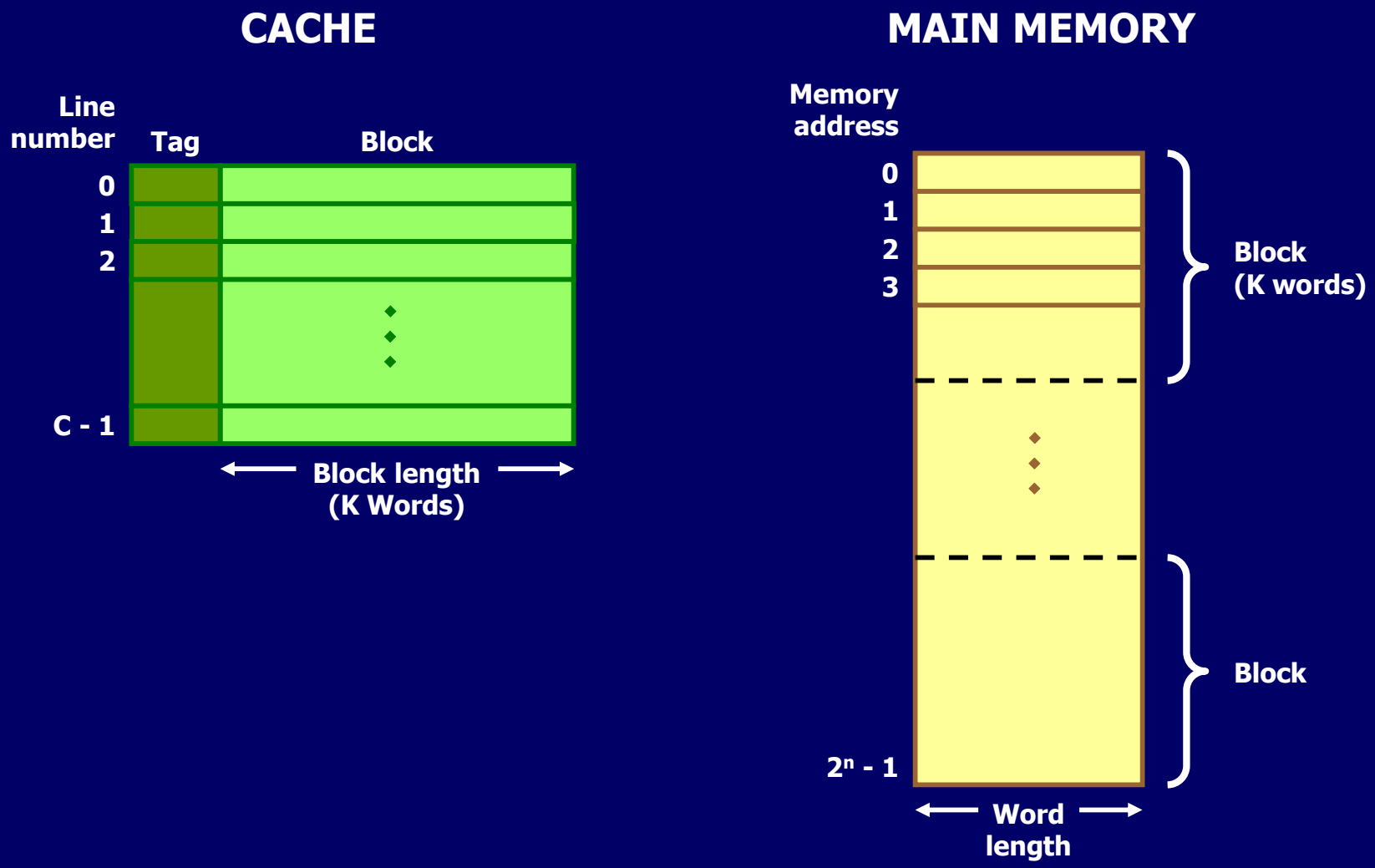


Cache/Main Memory Structure

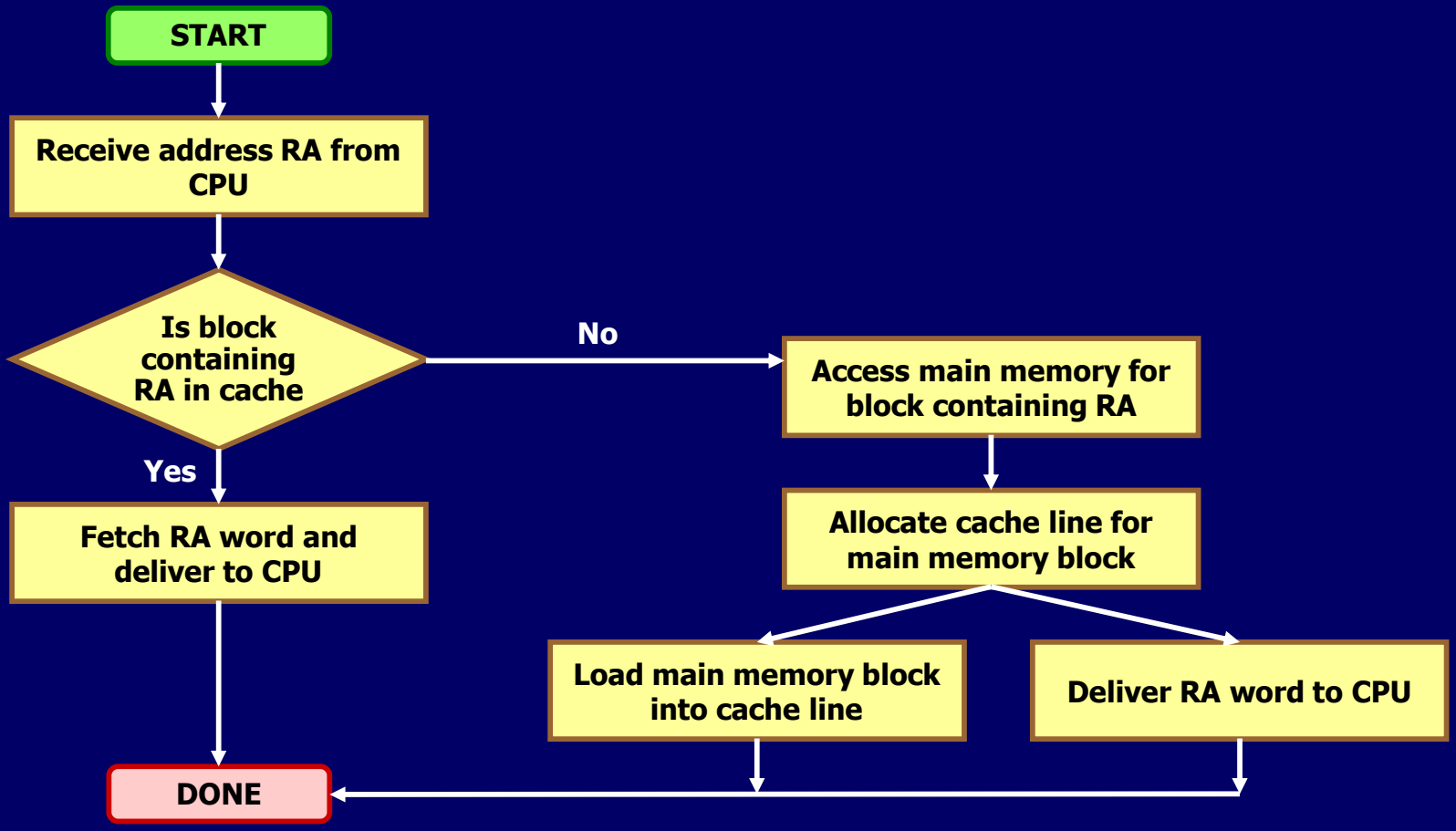
- Cache consists of C lines of K words each
  - Number of lines is much less than number of main memory blocks (C << M)
- Since there are more blocks than lines, an individual line cannot be permanently dedicated to a particular block
  - Cache includes tags to identify which block of main memory is in each line



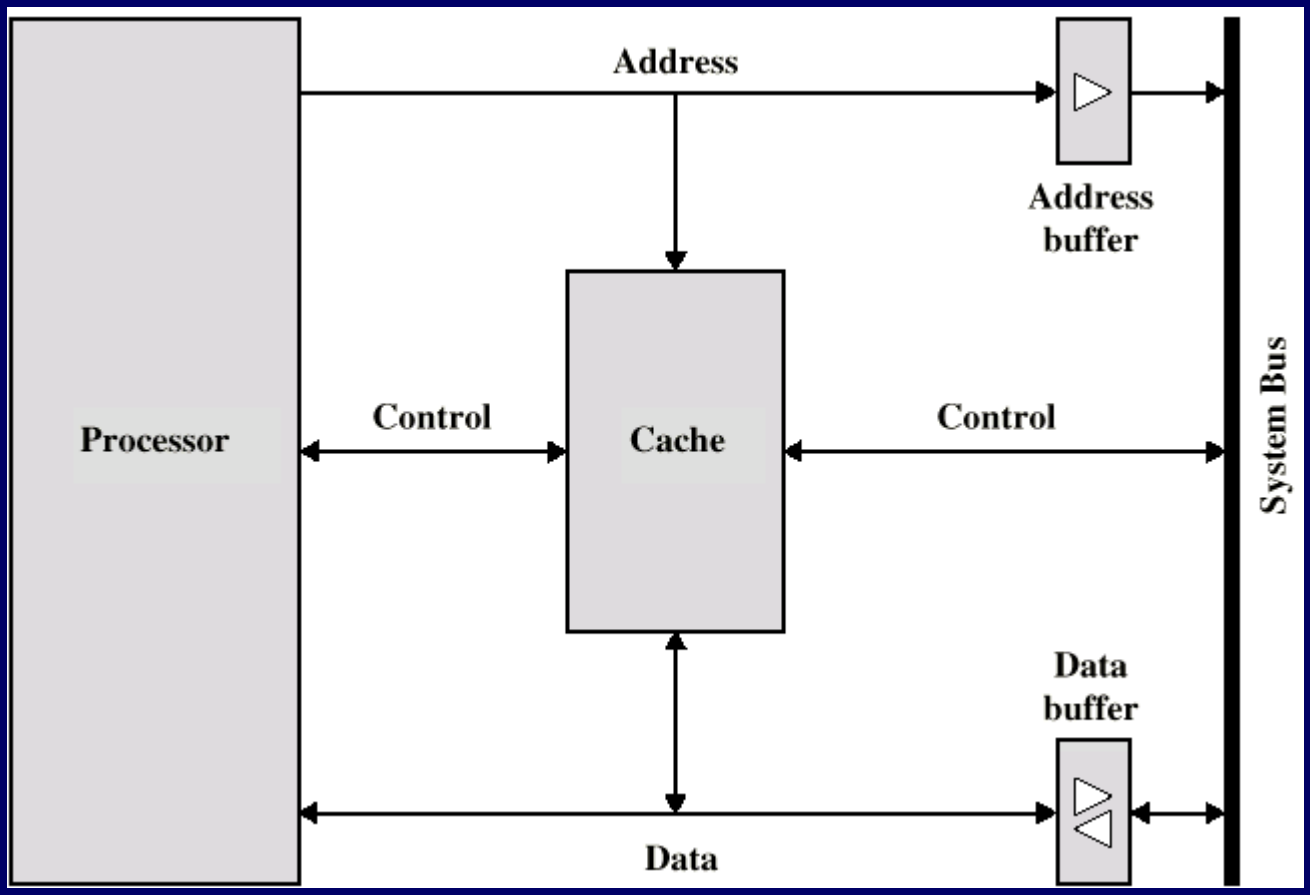
Cache/Main Memory Structure



Example: Cache Read Operation



Typical Cache Organization



Outline

- Cache Memory Principles
- Elements of Cache Design
  - Mapping function
  - Replacement algorithms
- Pentium 4/PowerPC caches

Cache Design

- Some design elements to classify and differentiate cache architectures
  - Size
  - Mapping Function
  - Replacement Algorithm
  - Write Policy
  - Block/Line Size
  - Number of Caches

Size Does Matter

- Ideal is
  - Size to be small enough so that overall average cost per bit is close to that of main memory
  - Large enough for overall average access time is close to that of cache alone
- Cost
  - More cache is expensive
- Speed
  - More cache is faster (up to a point)
  - Checking cache for data takes time

Size Does Matter

- Example cache sizes
  - Intel 80486 (1989)
    - L1: 8KB
  - Intel Pentium (1993)
    - L1: 8KB, L2: 256 - 512KB
  - Intel Pentium 4 (2000)
    - L1: 8KB, L2: 256KB

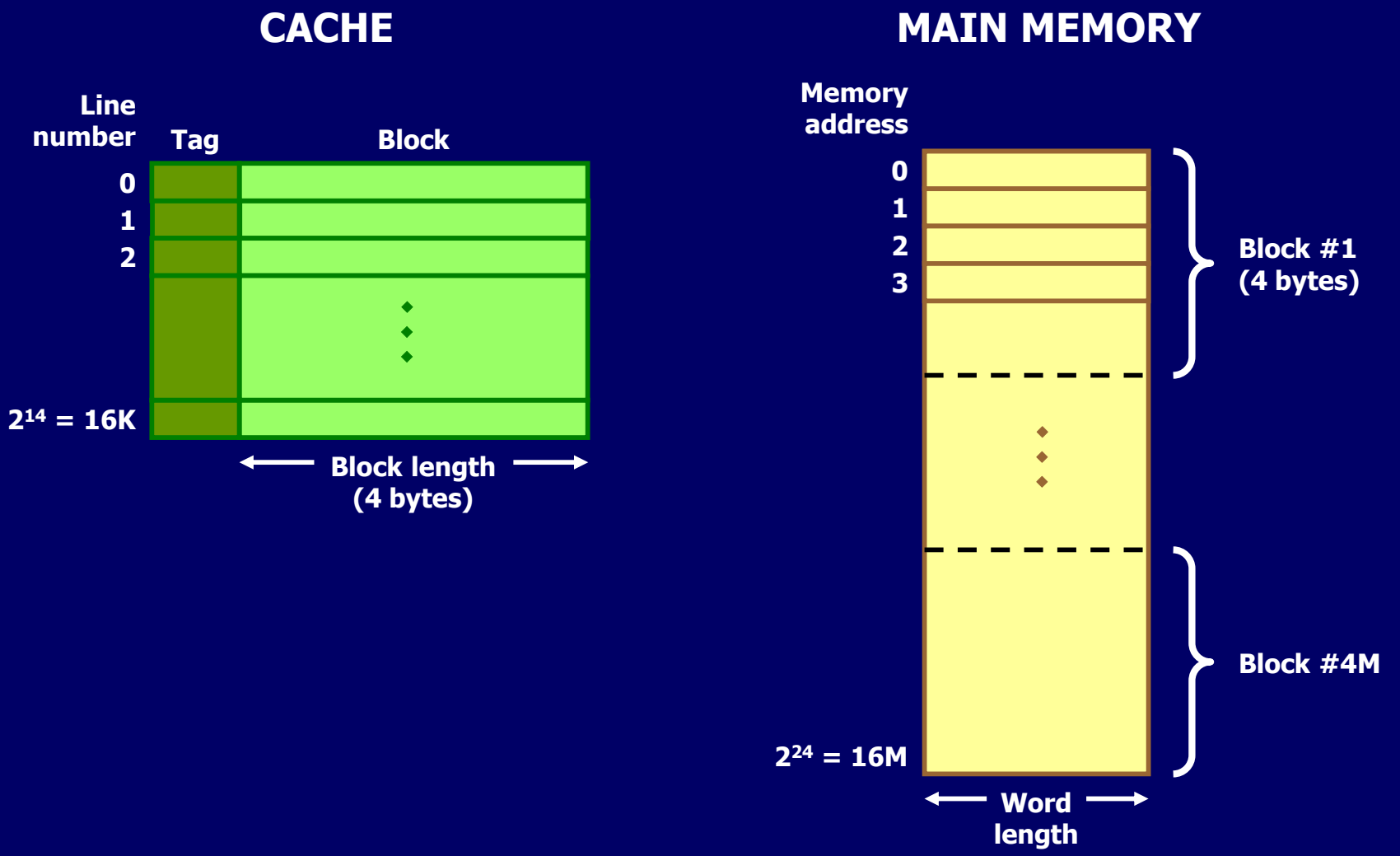
Mapping Function

- Fewer cache lines than main memory blocks
  - Algorithm needed for mapping main memory blocks into cache lines
- Need to determine which main memory block currently occupies a cache line
- Three mapping techniques
  - Direct
  - Associative
  - Set associative

Mapping Function

- Example situation:
  - Cache can hold 64KBytes
  - Data transferred in blocks of 4 bytes
    - i.e. cache is 16K (2^{14}) lines of 4 bytes
  - 16MBytes main memory
    - 24 bit address
    - (2^{24}=16M)
    - Can consider 4M blocks of 4 bytes each

Mapping Function



Direct Mapping

- Each block of main memory maps to only one cache line
  - i.e. if a block is in cache, it must be in one specific place
- Mapping (to a line number) expressed as
  - i = j modulo m
    - i = cache line number
    - j = main memory block number
    - m = number of lines in cache

Direct Mapping

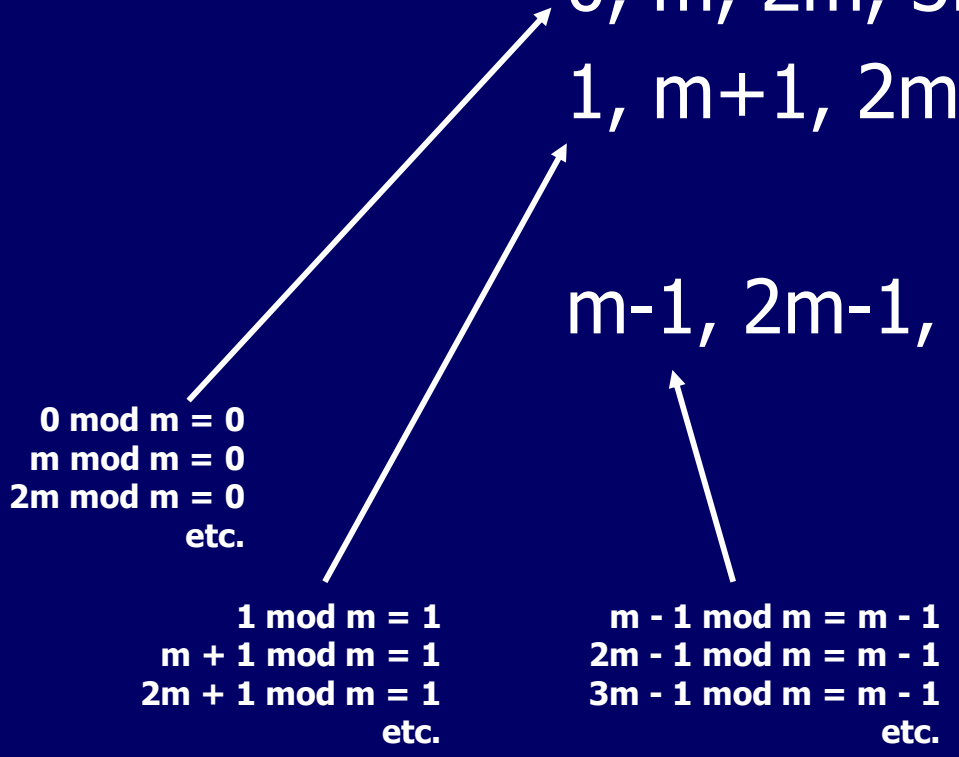
- Address is in two parts
  - Least Significant w bits identify unique word (within a block)
  - Most Significant s bits specify one memory block
    - Total of 2^s blocks of main memory
    - The MSBs are split into a cache line field r and a tag of s - r (most significant)
    - Total of $m = 2^r$ lines

Direct Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+ w}/2^{w} = 2^{s}$
- Number of lines in cache = m = 2^{r}
- Size of tag = (s – r) bits

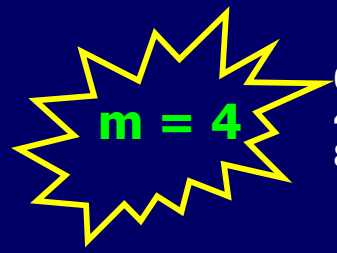
Direct Mapping Cache Line Table

Cache line	Main Memory blocks held
0	0, m, 2m, 3m … 2^s-m
1	1, m+1, 2m+1 … 2^s-m+1
m-1	m-1, 2m-1, 3m-1 … 2^s-1



Example: Direct Mapping Cache Line Table

Cache line	Main Memory blocks held
0	0, 4, 8, 12
1	1, 5, 9, 13
2	2, 6, 10, 14
3	3, 7, 11, 15



0 mod 4 = 0
4 mod 4 = 0
8 mod 4 = 0
etc.

1 mod 4 = 1
5 mod m = 1
9 mod m = 1
etc.

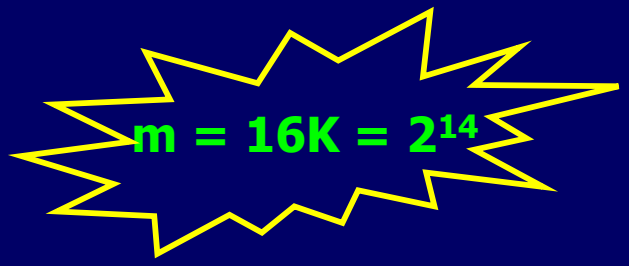
3 mod 4 = 3
7 mod 4 = 3
11 mod 4 = 3
etc.

Example: Direct Mapping Cache Line Table

Main memory block	Cache line
0	0
1	1
2	2
3	3
4	0
5	1
6	2
7	3
8	0
9	1

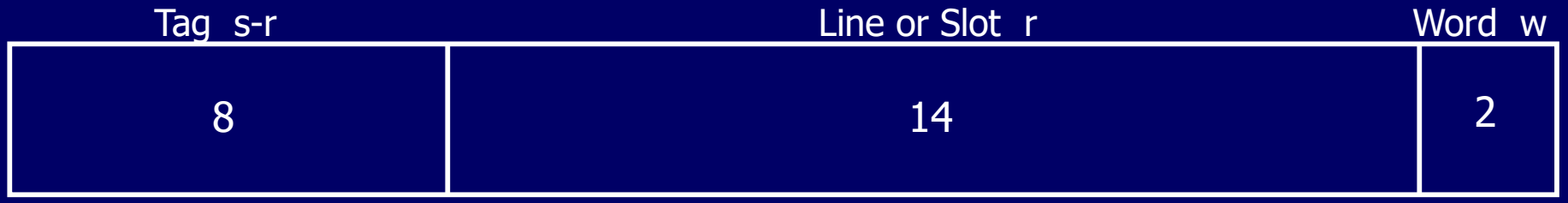
Direct Mapping Block Addressing

Cache line	Starting memory address of block
0	000000, 010000, …, FF0000
1	000004, 010004, …, FF0004
$2^{14} - 1$	00FFFC, 01FFFC, …, FFFFFC



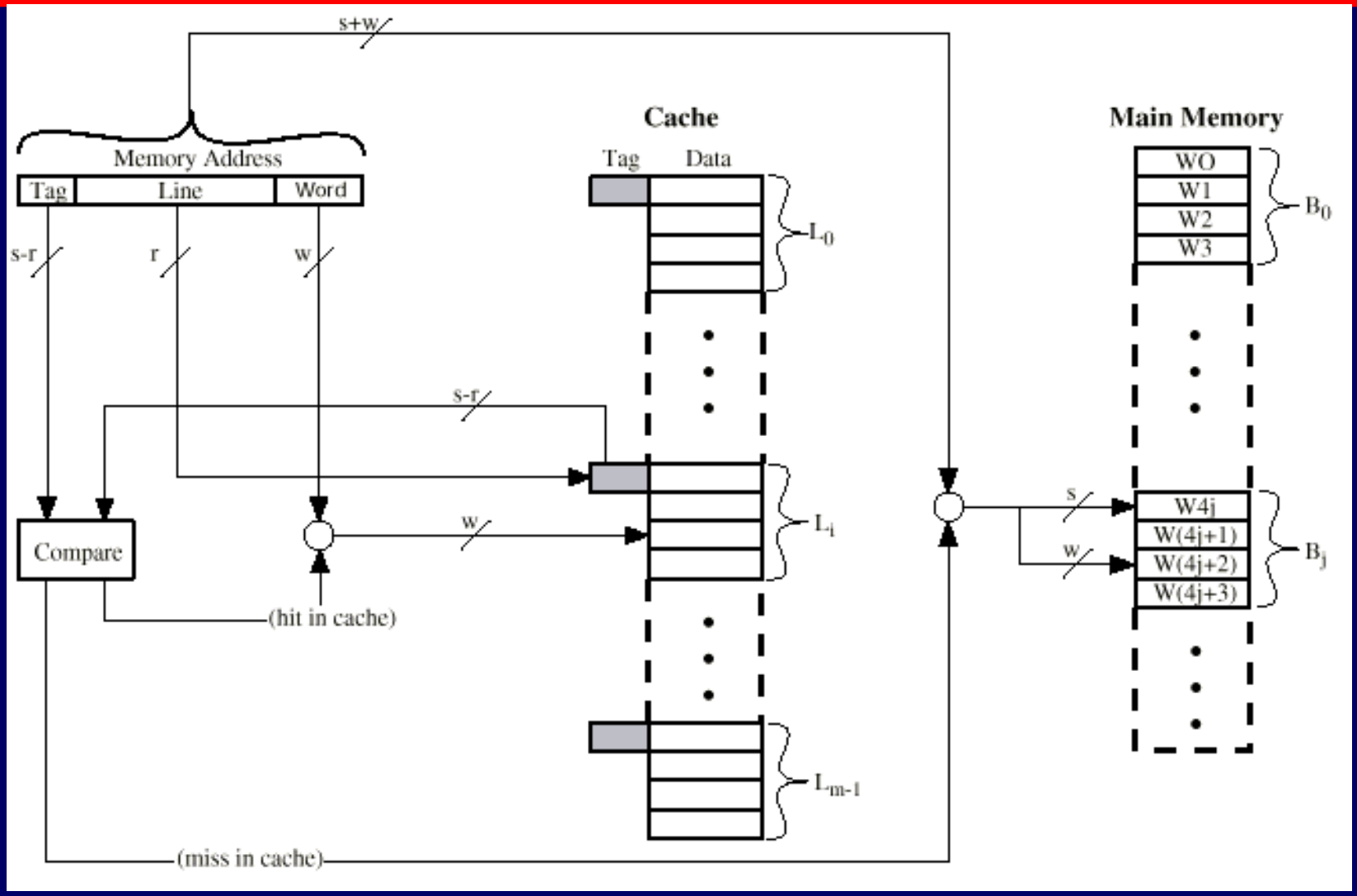
- No two blocks in the same line have the same tag

Direct Mapping Address Structure



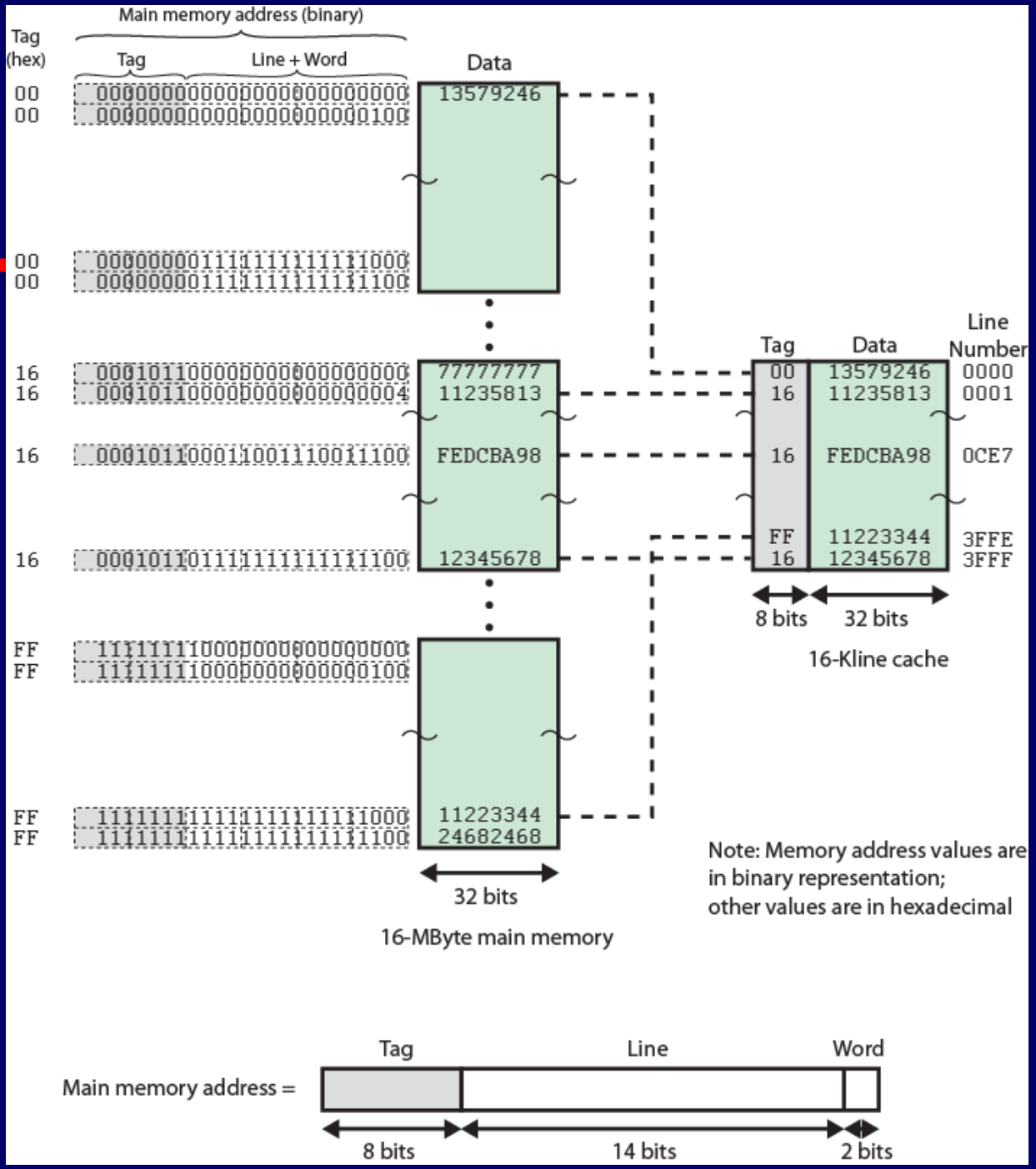
- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
  - 8 bit tag (= 22 - 14)
  - 14 bit slot or line
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

Direct Mapping Cache Organization



Direct Mapping

Example



Direct Mapping Pros & Cons

- Simple
- Inexpensive
- Fixed location for given block
  - If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high
    - Trashing

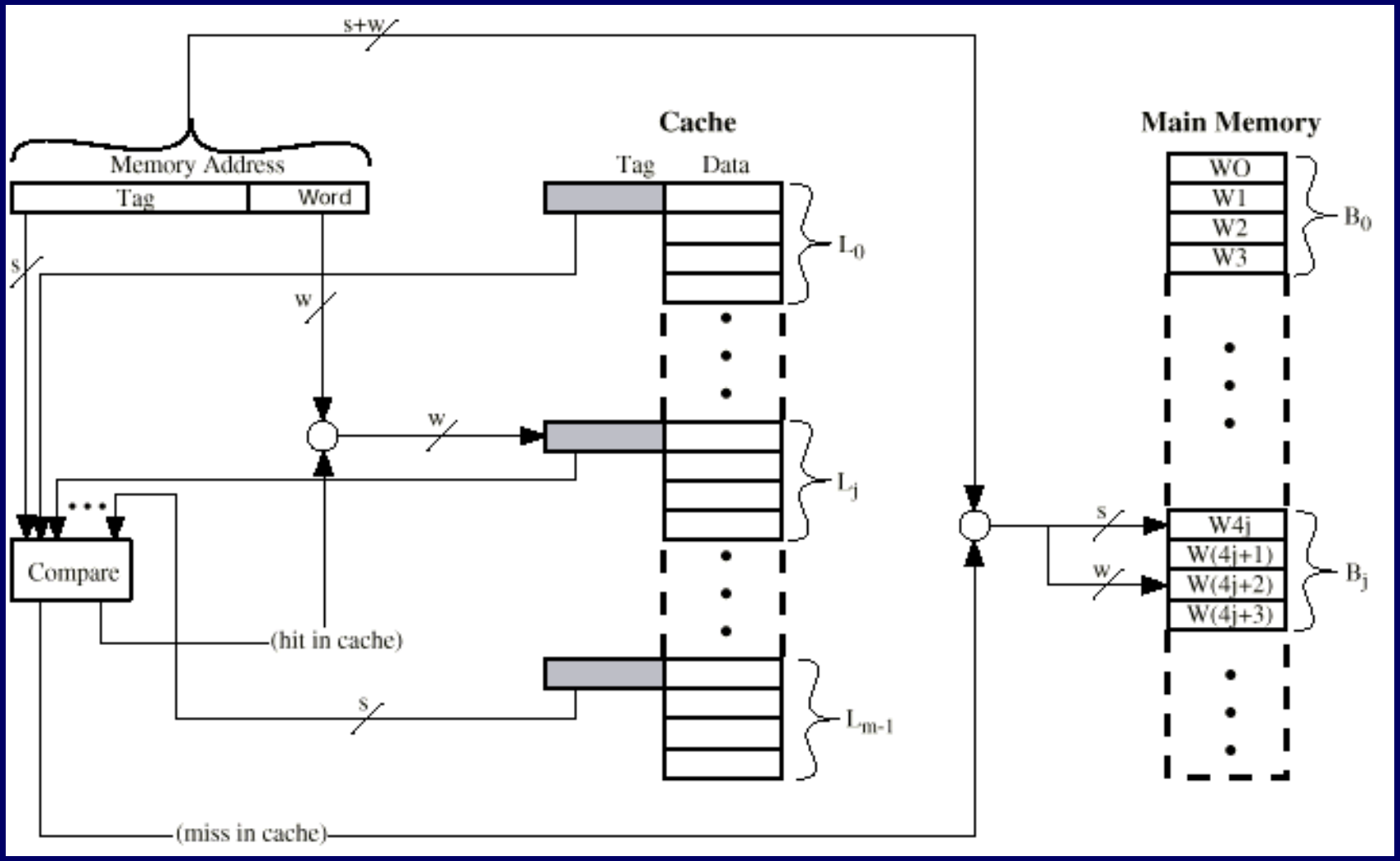
Associative Mapping

- A main memory block can load into any line of cache
- Memory address is interpreted as tag and word
- Tag uniquely identifies block of memory
- Every line's tag is examined for a match
- Cache searching gets expensive

Associative Mapping Summary

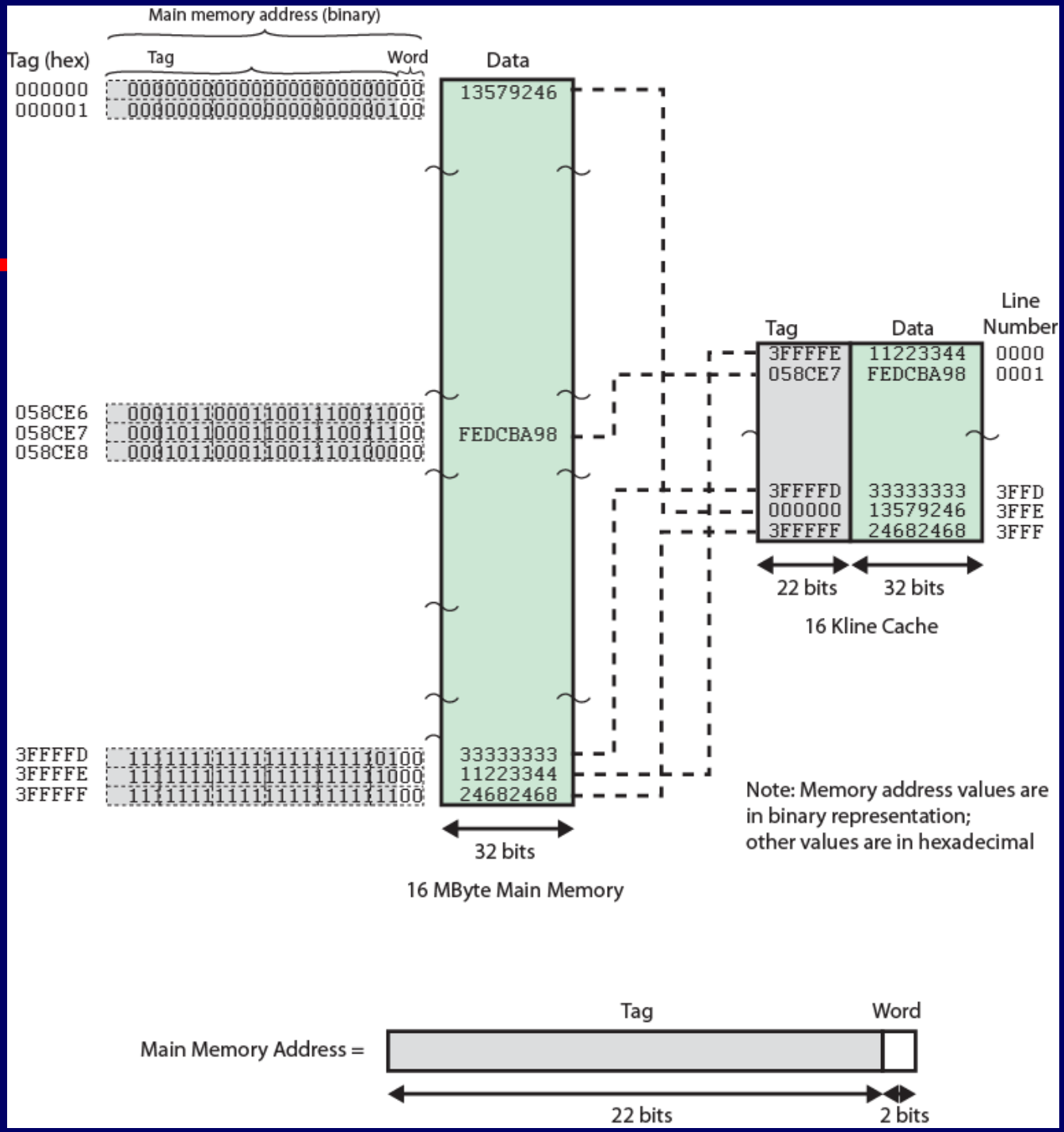
- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+ w}/2^{w} = 2^{s}$
- Number of lines in cache = undetermined
- Size of tag = s bits

Fully Associative Cache Organization



Associative Mapping

Example



Associative Mapping Address Structure

- 22 bit tag stored with each 32 bit block of data
- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify which 8 bit word is required from 32 bit data block
- e.g.
  - Address Tag Data Cache line
  - FFFFFC 3FFFFF 24682468 3FFF

Set Associative Mapping

- Cache is divided into a number of sets
- Each set contains a number of lines
- A given block maps to any line in a given set
  - e.g. Block B can be in any line of set i
- e.g. 2 lines per set
  - 2 way associative mapping
  - A given block can be in one of 2 lines in only one set
- Mapping (to a set number) expressed as
  - i = j modulo v
    - i = cache set number
    - j = main memory block number
    - v = number of sets in cache

Set Associative Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = 2^{s}
- Number of lines in set = k
- Number of sets = v = 2^{d}
- Number of lines in cache = kv = k * 2^{d}
- Size of tag = (s – d) bits

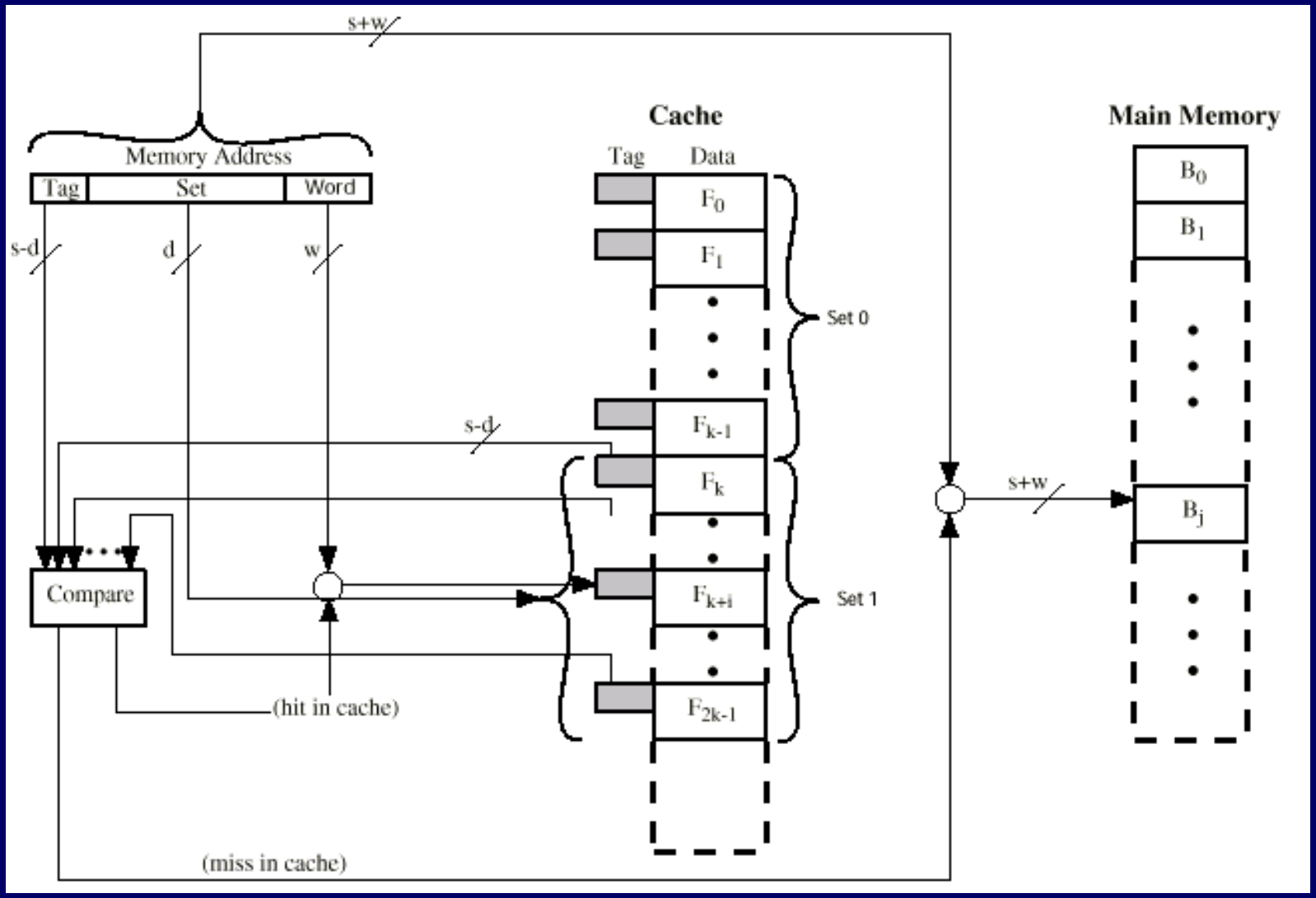
Set Associative Mapping Summary

- Extreme case
  - v = m, k = 1 (number of sets = number of lines, each set has 1 line)
    - = Direct mapping
  - v = 1, k = m (entire cache is one set, entire cache/set has m lines)
    - = Associative mapping
- Popular
  - v = m/2, k = 2 (number of sets = half of number of lines, each set has 2 lines)
  - v = m/4, k = 4
    - Slight improvement at higher cost

Set Associative Mapping Example

- 13 bit set number
- Block number in main memory is modulo 2^{13}
  - Recall: in direct mapping is modulo number of lines
- 000000, 008000, …, FF8000 … map to same set 0

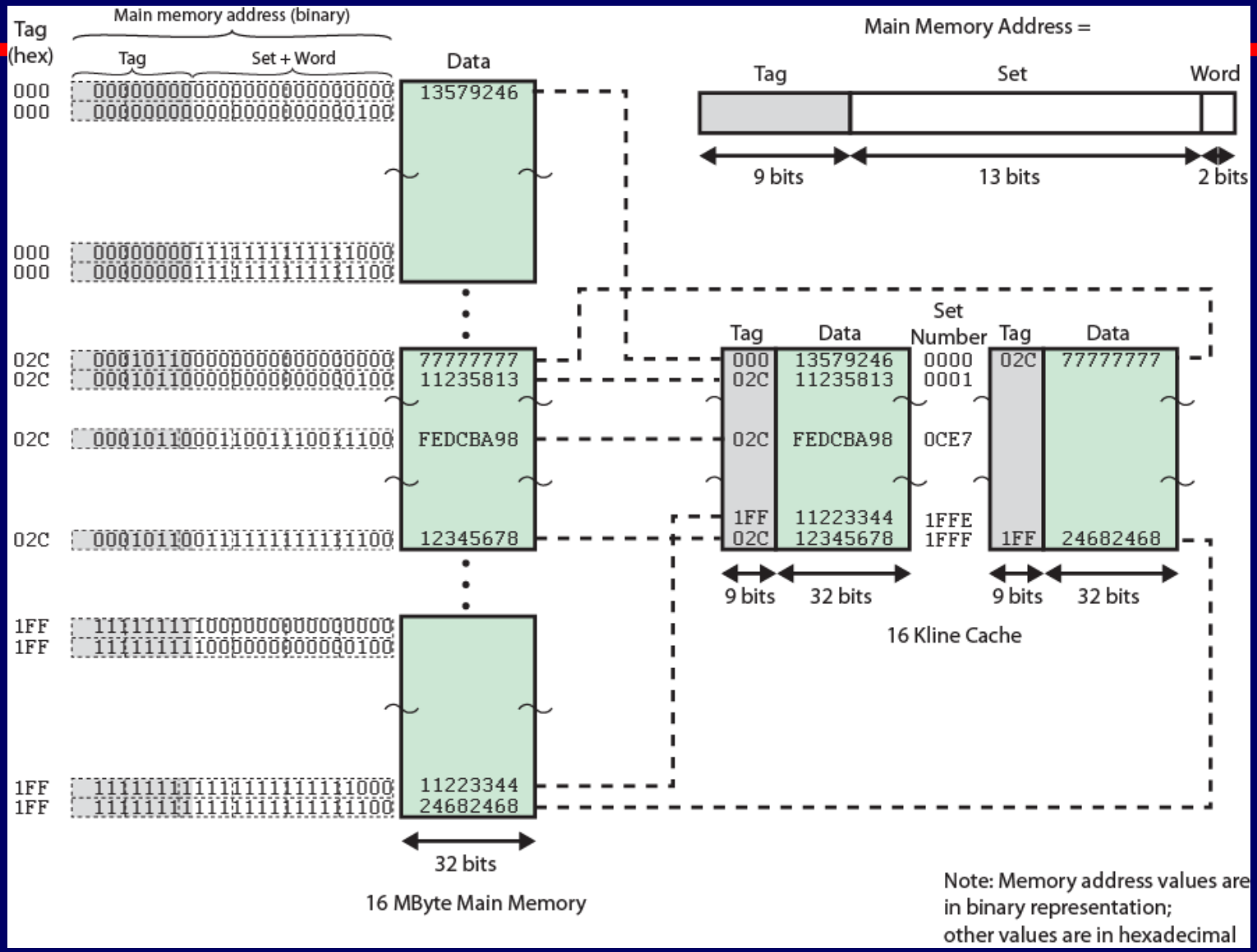
k-Way Set Associative Cache Organization



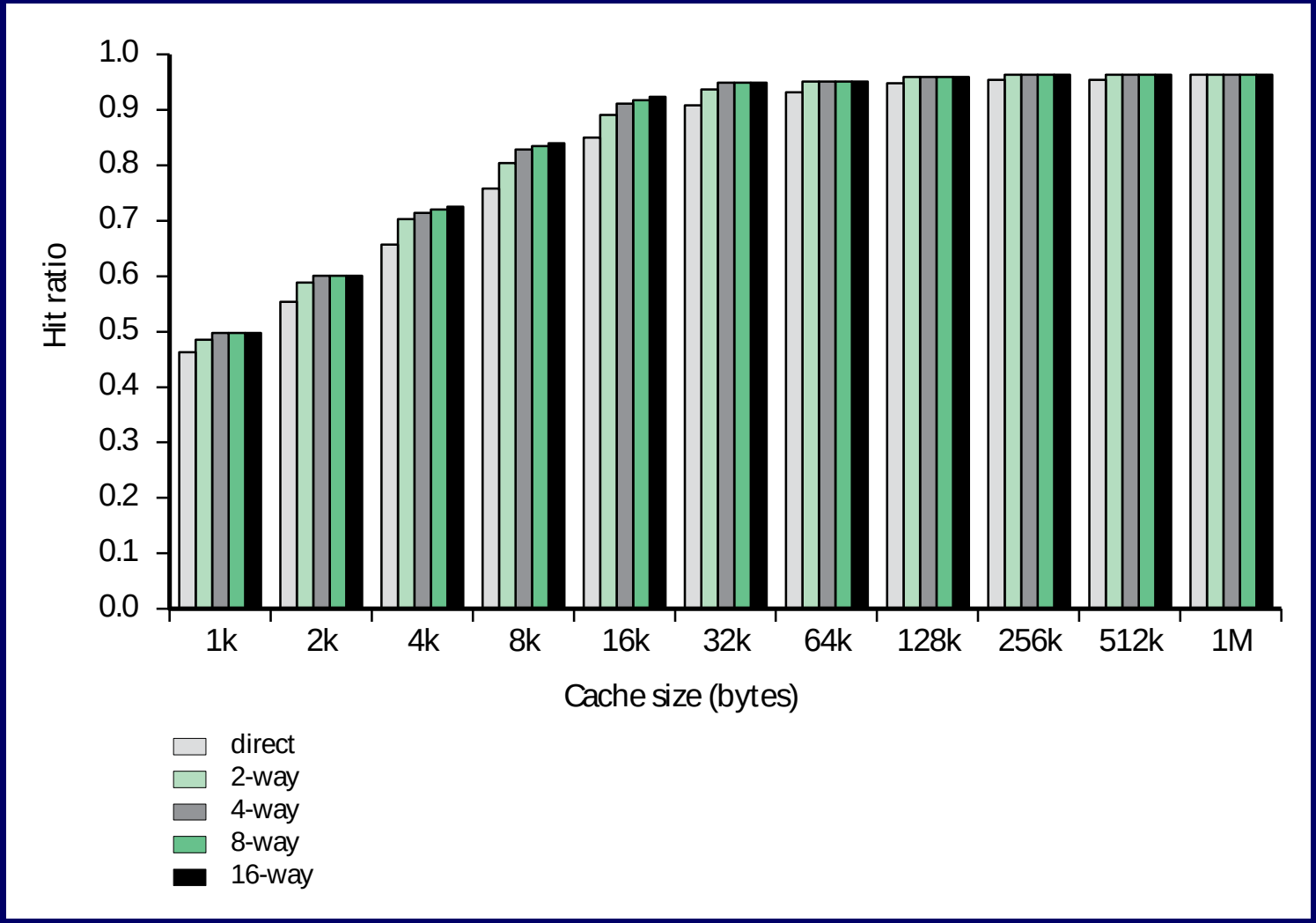
Set Associative Mapping Address Structure

- Use set field to determine cache set to look in
- Compare tag field to see if we have a hit
- e.g
  - Address Tag Data Set number
  - 1FF 7FFC 1FF 12345678 1FFF
  - 001 7FFC 001 11223344 1FFF

Two Way Set Associative Mapping Example



Varying Associativity over Cache Size



Replacement Algorithms (1) Direct mapping

- No choice
- Each block only maps to one line
- Replace that line

Replacement Algorithms (2) Associative & Set Associative

- Hardware implemented algorithm (speed)
- Least Recently used (LRU)
  - Each line has USE bit, set to 1 when accessed, the other line is set to 0
  - e.g. in 2-way set associative
    - Which of the 2 block is LRU?
- First in first out (FIFO)
  - Replace block that has been in cache longest
  - Implemented as round-robin/circular buffer

Replacement Algorithms (3) Associative & Set Associative

- Least frequently used
  - Replace block which has had fewest hits
  - Implemented by associating a counter with each line
- Random
  - Slightly inferior

Write Policy

- Before replacing a block in cache, check whether it has been altered
  - Must not overwrite a cache block unless main memory is up to date
- Two problems to consider on write policy
  - More than one device have access to main memory
  - Multiple CPUs may have individual caches
- Write policies
  - Write through
  - Write back

Write Through

- All writes go to main memory as well as cache
  - Always valid
- Multiple CPUs can monitor main memory traffic to keep local (to CPU) cache up to date
- Lots of traffic
- Slows down writes

Write Back

- Updates initially made in cache only
- Update bit for cache slot is set when update occurs
- If block is to be replaced, write to main memory only if update bit is set
- Problems
  - Portions of main memory are invalid (before write back)
  - I/O must access main memory through cache
- 15% of memory references are writes

Write Policy: More Issues

- When more than one device has a cache
  - If data in one cache are altered, this invalidates corresponding word in main memory, and the same word in other caches
  - Write through is not sufficient – other caches may contain invalid data
  - Approaches to maintain cache coherency
    - Bus watching with write through
      - All cache controllers monitors address lines. If another controller writes to a location that also resides in cache, that cache entry is invalidated
    - Hardware transparency
    - Noncacheable memory
      - Always cache miss. Always deal with main memory

Block / Line Size

- When block of data is placed in cache, it includes not only the desired word but also some adjacent words
- As block size increases, hit ratio increases
  - Principle of locality
  - More useful data are brought into cache
- When block size increase further, hit ratio decreases
  - Probability of using newly fetched information becomes less

Block / Line Size

- 2 specific effects
  - Larger blocks reduce number of blocks that fit into cache
    - Smaller number of (fewer) blocks results in data being overwritten shortly after being fetched
  - As a block becomes larger, each additional word is farther from the requested word
    - Less likely to be needed in future
- Relationship between block size and hit ratio is complex

Number of Caches

- Originally, the typical system has only a single cache
- Nowadays, multiple caches has become the norm
- 2 important aspects
  - Multilevel caches
  - Uniform vs split caches

Multilevel Caches

- It has now become possible to have an on-chip cache
  - Reduces processor's external bus activity
  - If requested data is found in the on-chip cache, bus access is eliminated
  - Speeds up execution times and increase performance
    - Shorter data paths internal to processor compared to bus lengths

Multilevel Caches

- If on-chip cache is good, then are off-chip caches (external caches) still desirable?
  - Yes
  - Results in two-level cache
    - Internal: Level 1 (L1)
    - External: Level 2 (L2)
  - Reason for L2
    - If no L2 cache and processor cannot find data in L1, processor must access DRAM or ROM across the bus
      - Slow memory access time and slow bus speed = poor performance
    - With L2, SRAM is used as L2
      - If SRAM is fast enough to match bus speed, then data can be accessed using a zero-wait state transaction

Multilevel Caches

- Modern multilevel caches have 2 noteworthy features
  - Many designs for off-chip L2 cache do not use system bus for transfer between L2 and processor (use separate data path)
  - With shrinking components, processors now incorporate L2 cache on the processor
- Potential savings due to L2 depends on hit rates on both L1 and L2
- Use of multilevel caches complicate design issues related to cache size, replacement algorithm and write policy

Unified vs Split Caches

- When on-chip cache was introduced, many designs consisted of a (unified) single cache to store data and instructions
- Recently, it is common to split cache into two
  - One for instructions and another for data
- Advantage of unified cache
  - Higher hit rate than split caches
    - Load between instruction and data is balanced automatically
    - Cache will fill up with either more instructions or data depending on execution pattern
  - Only one cache needs to be designed and implemented

Unified vs Split Caches

- Although unified cache is advantageous, trend is toward split caches
  - Particularly for superscalar machines (e.g. Pentium and PowerPC)
  - Advantage
    - Eliminates contention for the cache between instruction fetch/decode unit and execution unit
      - Important for design that relies on pipelining
      - E.g. execution unit performs memory access to load/store data and at the same time instruction prefetcher issues read request for an instruction

Outline

- Cache Memory Principles
- Elements of Cache Design
  - Mapping function
  - Replacement algorithms
- Pentium 4/PowerPC caches

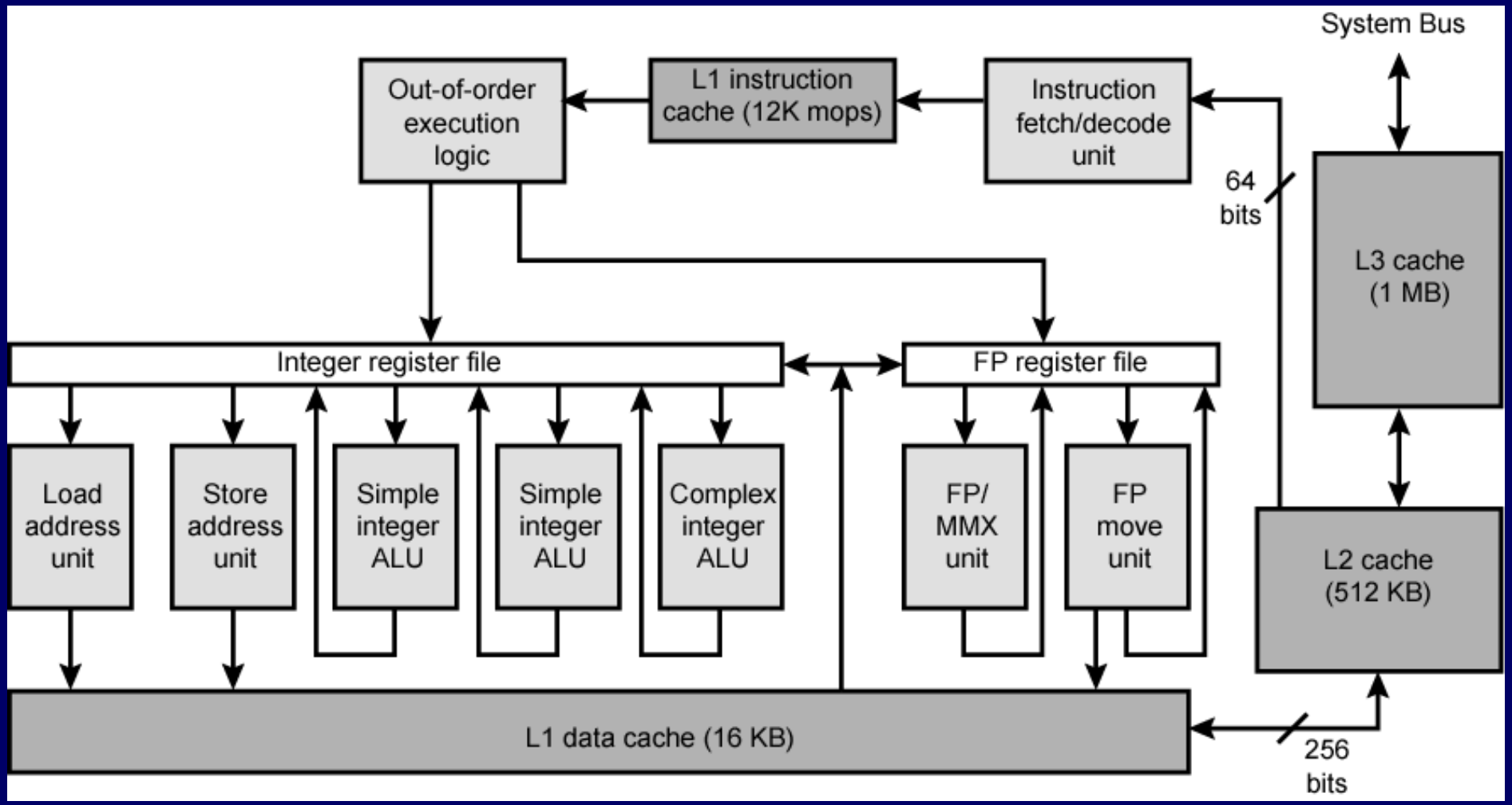
Pentium 4 Cache

- 80386: no on-chip cache
- 80486: 8k using 16 byte lines and four way set associative organization
- Pentium (all versions): two on-chip L1 caches
  - 1 for data & 1 for instructions (split cache)
- Pentium III: L3 cache added off-chip
- Pentium 4
  - L1 caches
    - 8k bytes, 64 byte lines, four way set associative
  - L2 cache
    - Feeding both L1 caches, 256k, 128 byte lines, 8 way set associative
  - L3 cache on chip

Intel Cache Evolution

Problem	Solution	Processor on which feature first appears
External memory slower than the system bus.	Add external cache using faster memory technology.	386
Increased processor speed results in external bus becoming a bottleneck for cache access.	Move external cache on-chip, operating at the same speed as the processor.	486
Internal cache is rather small, due to limited space on chip	Add external L2 cache using faster technology than main memory	486
Contention occurs when both the Instruction Prefetcher and the Execution Unit simultaneously require access to the cache. In that case, the Prefetcher is stalled while the Execution Unit's data access takes place.	Create separate data and instruction caches.	Pentium
Increased processor speed results in external bus becoming a bottleneck for L2 cache access.	Create separate back-side bus that runs at higher speed than the main (front-side) external bus. The BSB is dedicated to the L2 cache.	Pentium Pro
	Move L2 cache on to the processor chip.	Pentium II
Some applications deal with massive databases and must have rapid access to large amounts of data. The on-chip caches are too small.	Add external L3 cache.	Pentium III
	Move L3 cache on-chip.	Pentium 4

Pentium 4 Block Diagram



Pentium 4 Core Processor

- Fetch/Decode Unit
  - Fetches instructions from L2 cache
  - Decode into micro-ops
  - Store micro-ops in L1 cache
- Out of order execution logic
  - Schedules micro-ops
  - Based on data dependence and resources
  - May speculatively execute

Pentium 4 Core Processor

- Execution units
  - Execute micro-ops
  - Data from L1 cache
  - Results in registers
- Memory subsystem
  - L2 cache and systems bus

Pentium 4 Design Reasoning

- Decodes instructions into RISC like micro-ops before L1 cache
- Micro-ops fixed length
  - Superscalar pipelining and scheduling
- Pentium instructions long & complex
- Performance improved by separating decoding from scheduling & pipelining
- Data cache is write back
  - Can be configured to write through

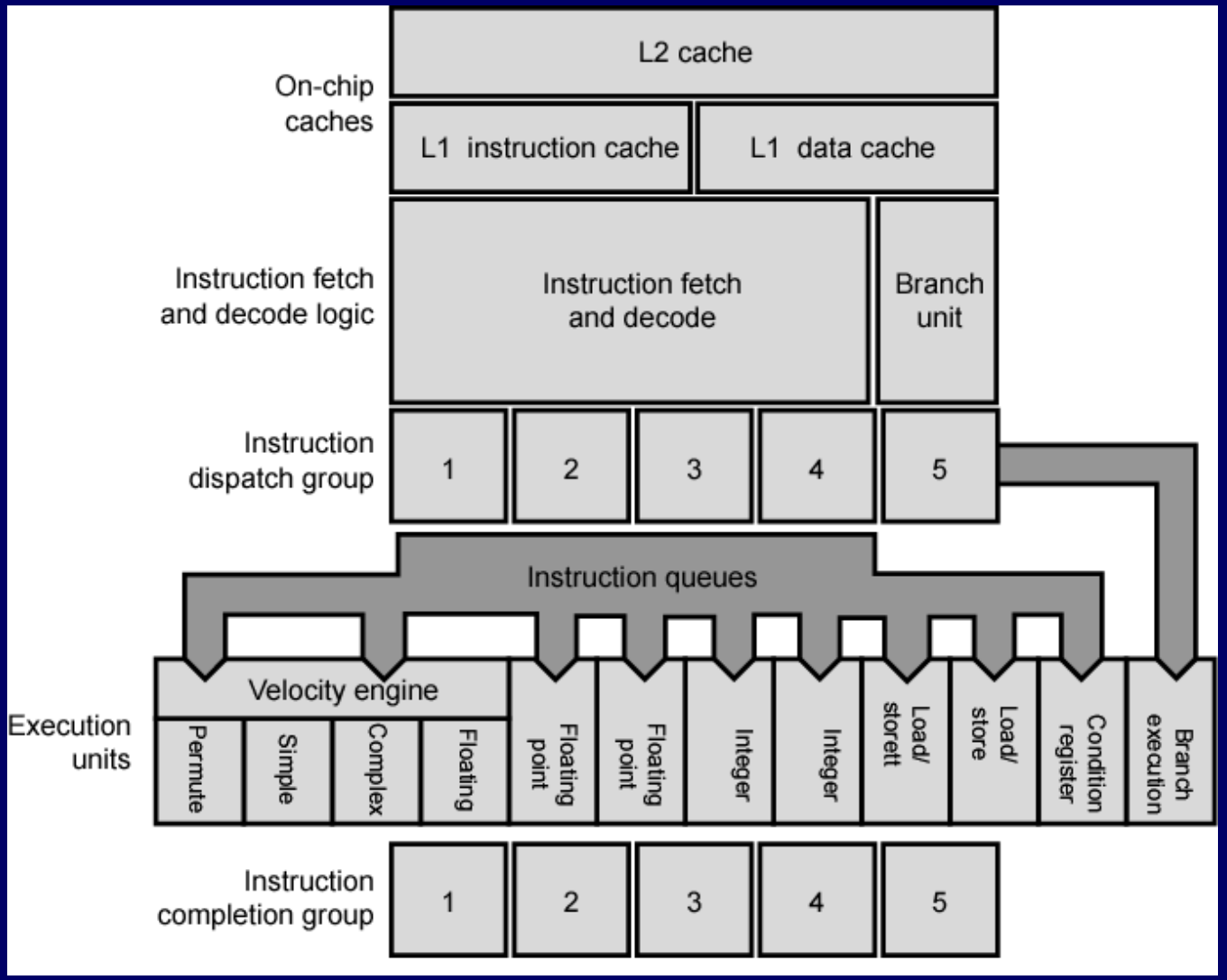
Pentium 4 Design Reasoning

- L1 cache controlled by 2 bits in register
  - CD = cache disable
  - NW = not write through
  - 2 instructions to invalidate (flush) cache and write back then invalidate
- L2 and L3 8-way set-associative
  - Line size 128 bytes

Power PC Cache Organization

- 601: single 32kb 8 way set associative
- 603: 16kb (2 x 8kb) two way set associative
- 604: 32kb
- 610: 64kb
- G3 & G4
  - 64kb L1 cache, 8 way set associative
  - 256k, 512k or 1M L2 cache, two way set associative
- G5
  - 32kB instruction cache
  - 64kB data cache

PowerPC G5 Block Diagram



Comparison of Cache Sizes

Processor	Type	Year of Introduction	L1 cache[a]	L2 cache	L3 cache
IBM 360/85	Mainframe	1968	16 to 32 KB	—	—
PDP-11/70	Minicomputer	1975	1 KB	—	—
VAX 11/780	Minicomputer	1978	16 KB	—	—
IBM 3033	Mainframe	1978	64 KB	—	—
IBM 3090	Mainframe	1985	128 to 256 KB	—	—
Intel 80486	PC	1989	8 KB	—	—
Pentium	PC	1993	8 KB/8 KB	256 to 512 KB	—
PowerPC 601	PC	1993	32 KB	—	—
PowerPC 620	PC	1996	32 KB/32 KB	—	—
PowerPC G4	PC/server	1999	32 KB/32 KB	256 KB to 1 MB	2 MB
IBM S/390 G4	Mainframe	1997	32 KB	256 KB	2 MB
IBM S/390 G6	Mainframe	1999	256 KB	8 MB	—
Pentium 4	PC/server	2000	8 KB/8 KB	256 KB	—
IBM SP	High-end server/ supercomputer	2000	64 KB/32 KB	8 MB	—
CRAY MTA[b]	Supercomputer	2000	8 KB	2 MB	—
Itanium	PC/server	2001	16 KB/16 KB	96 KB	4 MB
SGI Origin 2001	High-end server	2001	32 KB/32 KB	4 MB	—

[a] Two values seperated by a slash refer to instruction and data caches

[b] Both caches are instruction only; no data caches